

Chapter 10 Graph

A graph is a data structure that describes a binary relation between elements and has a webby looking graphical representation. Graph plays a significant role in solving a rich class of problems.

Definition:

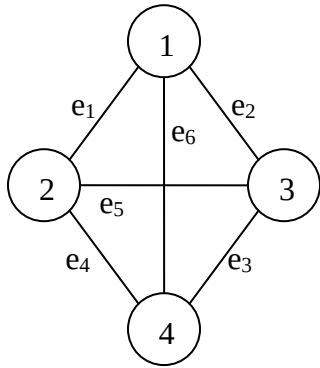
A graph $G = (V, E)$ consists of a finite set of non-empty set of vertices V and set of edges E .

$$V = \{v_1, v_2, \dots, v_n\}$$

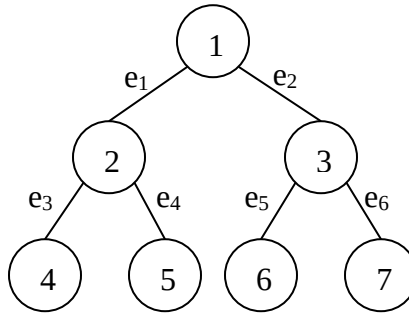
$$E = \{e_1, e_2, \dots, e_n\}$$

Each edge e is a pair (v, w) where $v, w \in V$. The edge is also called arc.

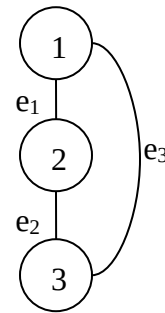
Example of graphs:



G_1



G_2



G_3

$$V(G_1) = \{1, 2, 3, 4\}$$

$$E(G_1) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$$

$$e_1 = (1, 2)$$

$$e_2 = (1, 3)$$

$$e_3 = (3, 4)$$

$$e_4 = (2, 4)$$

$$e_5 = (2, 3)$$

$$e_6 = (1, 4)$$

$$V(G_2) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G_2) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$$

$$e_1 = (1, 2)$$

$$e_2 = (1, 3)$$

$$e_3 = (2, 4)$$

$$e_4 = (2, 5)$$

$$e_5 = (3, 6)$$

$$e_6 = (3, 7)$$

$$V(G_3) = \{1, 2, 3\}$$

$$E(G_3) = \{e_1, e_2, e_3\}$$

$$e_1 = (1, 2)$$

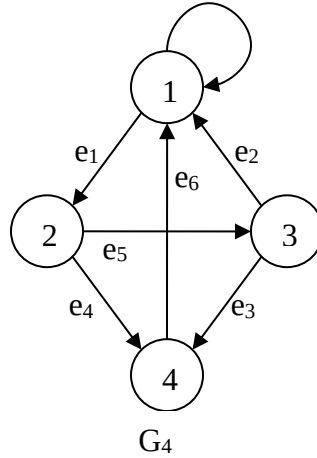
$$e_2 = (2, 3)$$

$$e_3 = (1, 3)$$

The vertices are represented by points or circles and the edges are line segments connecting the vertices. If the graph is directed, then the line segments have arrow heads indicating the direction.

Directed Graphs:

If every edge (i, j) , in $E(G)$ of a graph G is marked by a direction from i to j , then the graph is called directed graph. It is often called digraph.



In edge $e = (i, j)$, we say, e leaves i and enters j . In digraphs, self loops are allowed. The indegree of a vertex v is the number of edge entering v . The outdegree of a vertex v is the number of edges leaving v . The sum of the indegrees of all the vertices in a graph equals to the sum of outdegree of all the vertices.

$$\sum_{i=0}^n d_{in}(v_i) = \sum_{i=0}^n d_{out}(v_i) \quad n = \text{no. of vertices}$$

Undirected Graph:

If the directions are not marked for any edge, the graph is called undirected graph. The graphs G_1, G_2, G_3 are undirected graphs. In an undirected graph, we say that an edge $e = (u, v)$ is incident on u and v (u and v are connected).

Undirected graph don't have self loops. Incidence is a symmetric relation i.e. if $e = (u, v)$ Then u is a neighbor of v and vice versa. The degree of a vertex, $d(v)$, is the total number of edges incident on it.

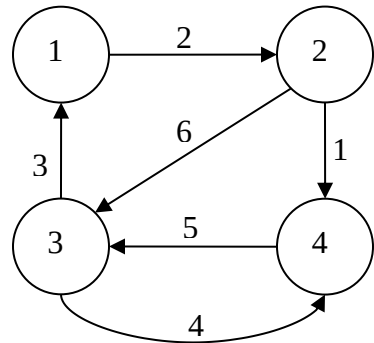
$$\sum_{i=0}^n d(v_i) = 2|E|$$

The sum of the degrees of all the vertices in a graph equals twice the number of edges if $d(v) = 0$, v is isolated. If $d(v) = 1$, v is pendent.

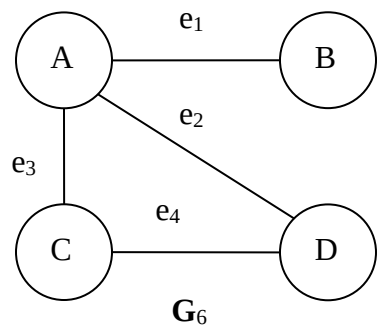
Weighted Graphs:

A graph is said to be weighted graph if every edge in the graph is assigned some weight or value. The weight is a positive value that may represent the cost of moving along the edge, distance between the vertices etc.

The two vertices with no edge (path) between them can be thought of having an edge (path) with weight infinite.

**Additional Terminology in Graph****Adjacent and Incident**

Two vertices i and j are called adjacent if there is an edge between the two. Eg. The vertices adjacent to vertex A, in the fig below are B, C, D. The adjacent vertices of C are A and D.



If $e(i, j)$ is an edge on $E(G)$, then we say that the edge $e(i, j)$ is incident on vertices i and j . Eg. in the above figure, e_4 is incident on C and D. If (i, j) is directed edge, then i is adjacent to j and j is adjacent from i .

Path:

A path is a sequence of distinct vertices, each adjacent to the next. For e.g. the sequence of vertices e_1, e_3, e_4 (i.e. (B,A), (A,C), (C,D)) of the above graph form a path from B to D. The length of the path is the number of edges in the path. So, the path from B to D has length equal to three.

In a weighted graph, the cost of a path is the sum of the costs of its edges. Loops have path length of 1.

Cycle:

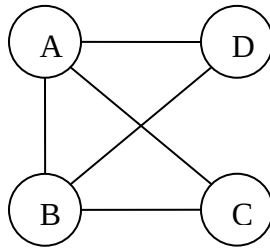
A cycle is a path containing at least three vertices such that the last vertex on the path is adjacent to the first. In the above graph G_6 , A, C, D, A is a cycle.

Connected:

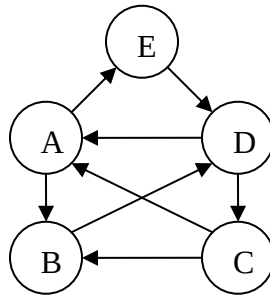
Any graph is *connected* provided there exists a path (directed or undirected) between any two nodes. A digraph is said to be *strongly connected* if, for any two vertices there is a direct path from i to j .

A digraph is said to be weakly connected if, for any two vertices i and j , there is a direct path from i to j or j to i . or A *weakly-connected graph* is a directed graph for which its underlying undirected graph is connected.

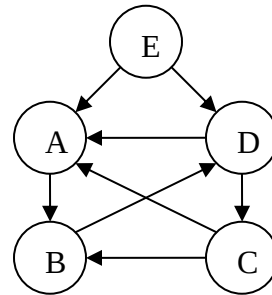
Example



Connected



Strongly Connected



Weakly Connected

Complete graph

A *complete graph* is a graph in which there is an edge between every pair of vertices.

Trees and Graph

A tree is a connected graph with no cycle. A tree has $|E| = |V| - 1$ edges. Since, it is connected, there is a path between any two vertices.

Representation of Graphs

A graph can be represented in many ways. Some of them are described in this section.

Adjacency matrix

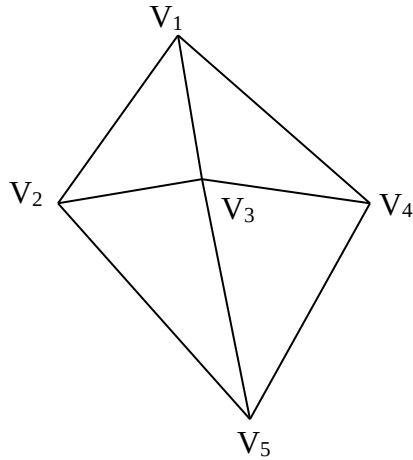
Adjacency matrix A for a graph $G = (V, E)$ with n vertices, is $n \times n$ matrix, such that

$A_{ij} = 1$, if there is an edge from v_i to v_j

$A_{ij} = 0$, if there is no such edge

We can also write,

$$A(i, j) = \begin{cases} 1 & \text{if and only if } (v_i, v_j) \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

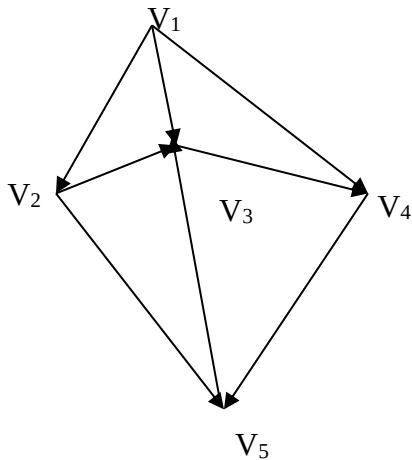


Eg; An adjacency matrix for the following undirected graph is

$A =$

	V ₁	V ₂	V ₃	V ₄	V ₅
V ₁	0	1	1	1	0
V ₂	1	0	1	0	1
V ₃	1	1	0	1	1
V ₄	1	0	1	0	1
V ₅	0	1	1	1	0

If the graph is directed, then the adjacency matrix will be as follows. The number in each row tells the outdegree of that vertex.



$A =$

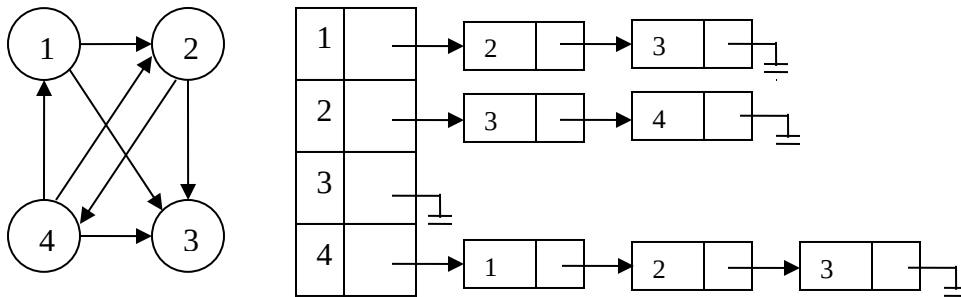
	V ₁	V ₂	V ₃	V ₄	V ₅
V ₁	0	1	1	1	0
V ₂	0	0	1	0	1
V ₃	0	0	0	1	0
V ₄	0	0	0	0	1
V ₅	0	0	1	0	0

In this representation we require n^2 bits to represent a graph with n nodes. It is a simple way to represent a graph, but it has following disadvantages:

- it takes $O(n^2)$ space
- it takes $O(n^2)$ time to solve most of the problems.

Adjacency List Representation

The n rows of an adjacency matrix can be represented as n linked lists. This is one list for each node in a graph. Each list will contain adjacent nodes. Each node has two fields, *Vertex* and *Link*. The *Vertex* field of a node p will contain the nodes that are adjacent to the node p .



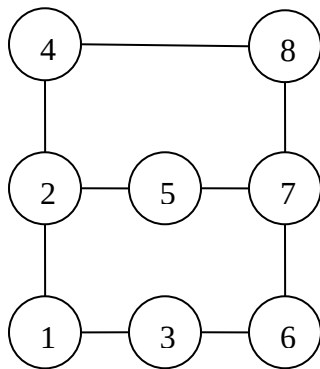
Graph Traversals

A graph traversal means visiting all the nodes of the graph. Basically, there are two methods of graph traversals.

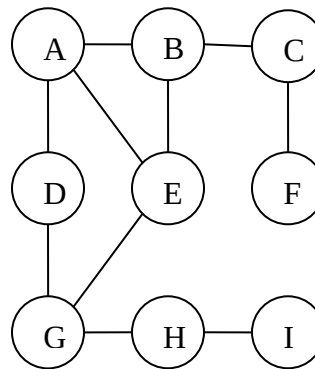
1. Breadth First Traversal
2. Depth First Traversal

1. Breadth First Traversal

The *breadth first traversal* begins with a given vertex and then it next visits all the vertices adjacent to v , putting the vertices adjacent to these in a waiting list to be traversed after all vertices adjacent to v have visited. Breadth First Traversal uses queue.



Starting from 1 Breadth First Traversal yields 1, 2, 3, 4, 5, 6, 8, 7



Starting from A Breadth First Traversal yields A, B, D, E, C, G, F, H, I

Algorithm:

```

void BFTraversal(Graph G)
{
    for each vertex v in G
        visited[v] = false;
    for each vertex v in G
        if(visited[v]==false)
        {
            enqueue(V, Q);
            do
            {
                v=dequeue(q);
                visited[v]=true;
                visited(v);
                for each vertex w adjacent to v
                    if(visited[w]==false)
                        enqueue(w, q);
            }while(isempty(Q)==false)
        }
    }
}

```

2. Depth First Traversal

The *Depth First Traversal* algorithm is roughly analogous to preorder tree traversal, the algorithm follows:

suppose that the traversal has just visited a vertex v , and let $w_0, w_1, \dots, w_k$ be the vertices adjacent to v

- next, visit w_0 and keep $w_1, \dots, w_k$ waiting
- after visiting w_0 , we traverse all the vertices to which it is adjacent before returning to traverse $w_1, \dots, w_k$.

Depth first traversal uses stack to store the adjacent vertices but one.

For above 2 figures Depth First Traversal yields

Adjacency list		Adjacency list	
1	2, 3	A	B, D, E
2	1, 4, 5	B	A, C, E
3	1, 6	C	B, F
4	2, 8	D	A, G
5	2, 7	E	A, B, G
6	3, 7	F	C
7	5, 6, 8	G	D, E, H
8	4, 7	H	G, I
		I	H
DF Traversal:		DF Traversal:	
	1, 2, 4, 8, 7, 5, 6, 3		A, B, C, F, E, G, D, H, I

Algorithm:

```

void DFTraversal(Graph G)
{
    for each vertex v in G
        Visited[v]=false;

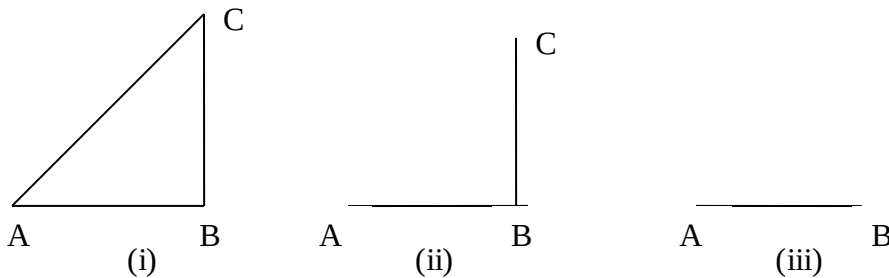
    for each vertex v in G
        If(visited[v]==false)
            Traverse(v);
}

void Traverse(Vertex v)
{
    Visited[v]=true;
    Visited(v);
    For each vertex w adjacent to v
    {
        If(visited[w]==false)
            Traverse(w);
    }
}

```

Subgraph:

Let H be a graph with vertex set $V(H)$ and edge set $E(H)$ and, similarly, let G be a graph with vertex set $V(G)$ and edge set $E(G)$. Then H is said to be a subgraph of G, written as $H \leq G$ if $V(H) \leq V(G)$ and $E(H) \leq E(G)$.

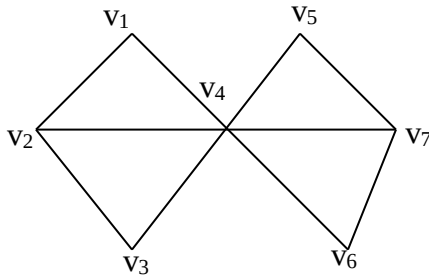


Here, second and third graph are subgraph of the first.

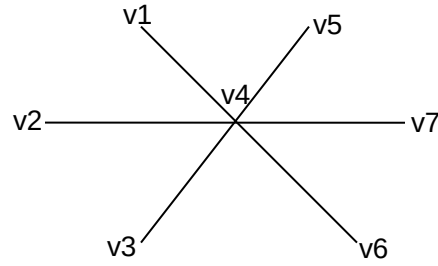
Spanning Tree:

Let a graph $G=(V,E)$ be a graph. If T is a subgraph of G and contains all the vertices but no cycles/circuits, then 'T' is said to be spanning tree of G.

In other words, the spanning tree of an undirected graph G is the tree formed from graph edges which connects all the vertices of G.

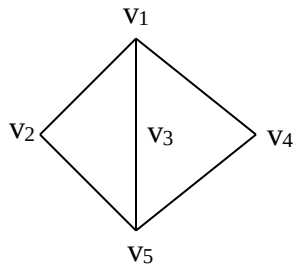


Undirected Graph G

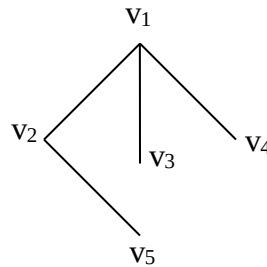


Spanning Tree of G

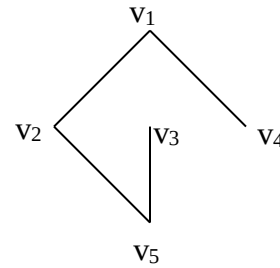
For a given graph $G=(V, E)$, if Breadth-First search and Depth-First search call is made, then the resultant tree that is generated is the spanning tree of the graph G. Thus, by making a Depth-First search on G, we get the Depth-First spanning tree. Similarly, on applying Breadth-First search on graph 'G' we get Breadth First spanning tree.



Graph G



BF Spanning Tree and DF Spanning Tree of G

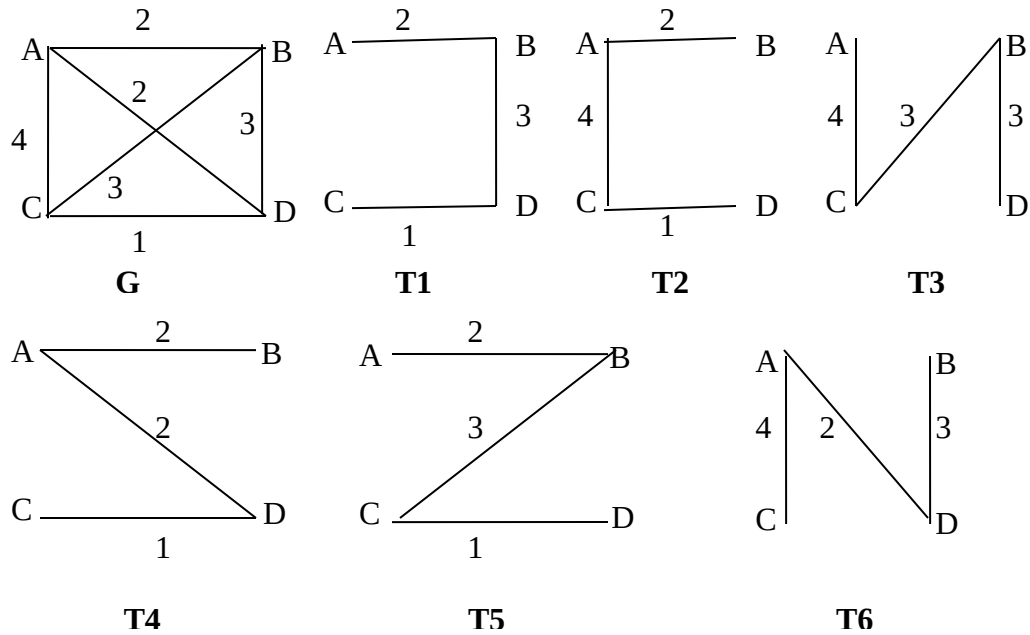


Minimum Spanning Tree

The cost of a spanning tree of a weighted, undirected graph is the sum of the costs (weights) of the edges in tree.

The minimum spanning tree of a weighted undirected graph is the spanning tree that connects all the vertices in G at lowest cost. So, if T is the minimum spanning tree of graph G and T' is any other spanning tree of G then,

$$W(T) \leq W(T')$$



Here, T_1 to T_6 are the *spanning trees* of G . Among them, T_4 has minimum cost (i. e. 5). So, T_4 is the minimum spanning tree of graph G

There are several algorithms available to determine the *minimal spanning tree* of a given weighted graph.

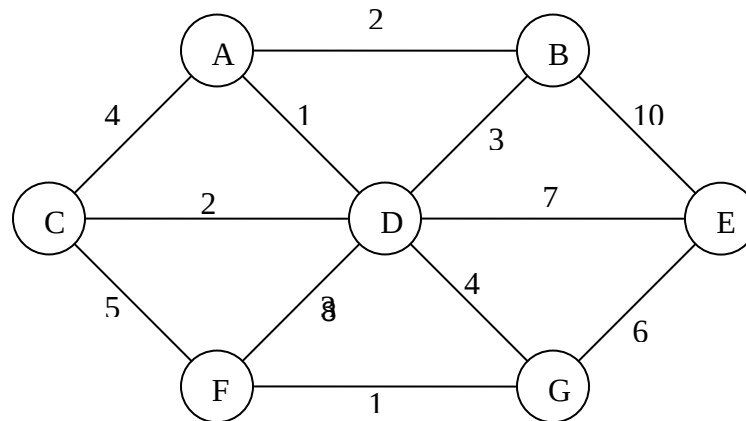
Kruskal's Algorithm

To determine minimum spanning tree consider a graph G with n vertices.

Step 1: List all the edges of the graph G with increasing weights

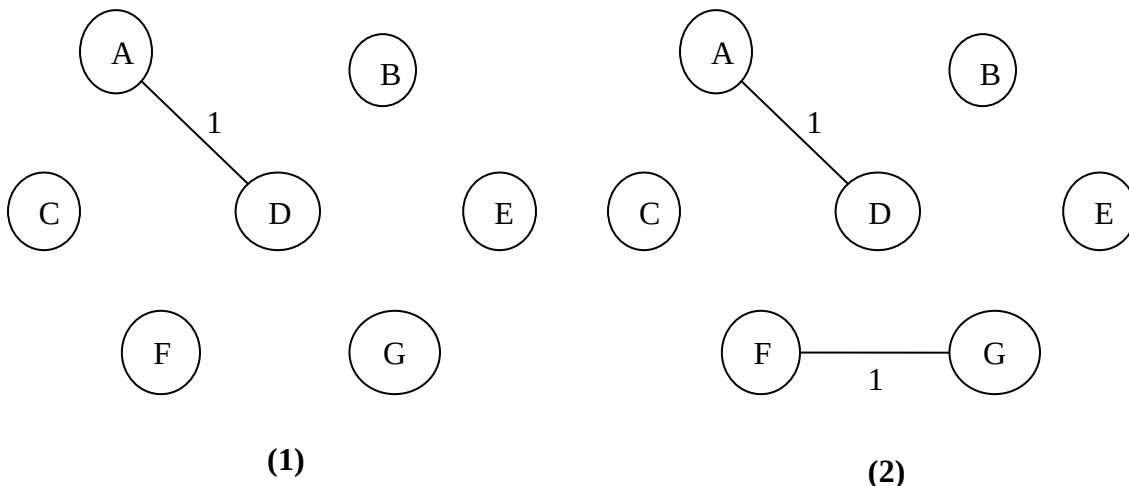
Step 2: Proceed sequentially to select one edge at a time joining n vertices of G such that no cycle is formed until $n-1$ edges are selected.

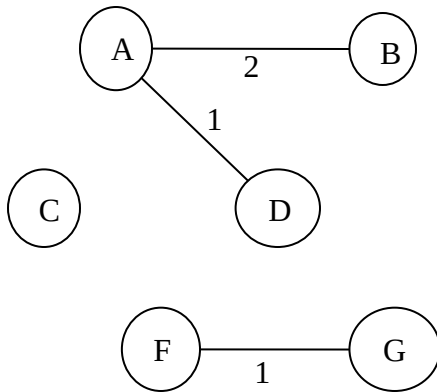
Step 3: Draw the $n-1$ edges that were selected forming a minimal spanning tree T of G .



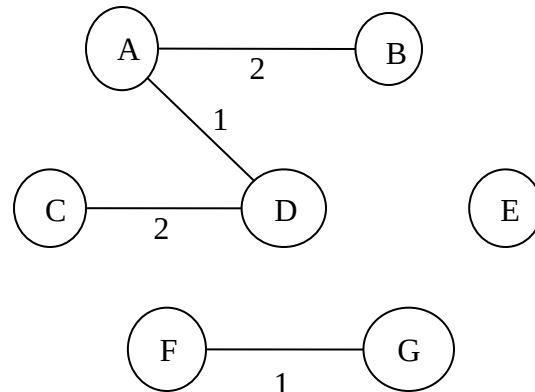
We have,

Edges:	AD	FG	AB	CD	BD	AC	DG	CF	EG	DE	DF	BE
Weights	1	1	2	2	3	4	4	5	6	7	8	10

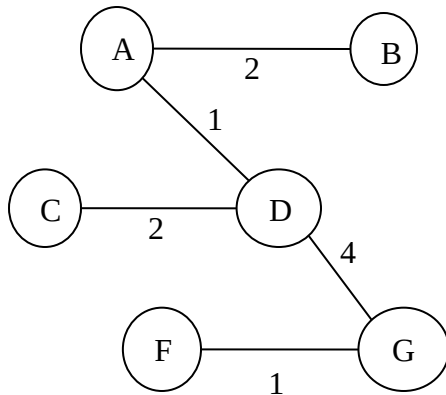




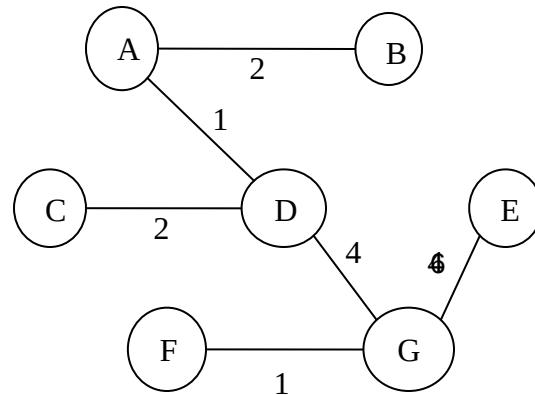
(3)



(4)



(5)



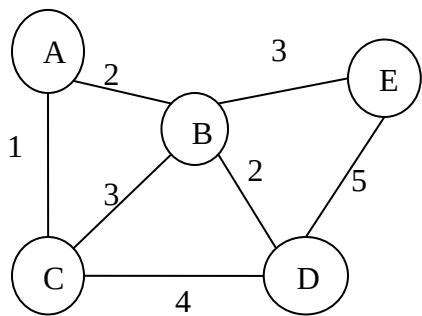
(6)

Hence, the minimum spanning tree is generated.

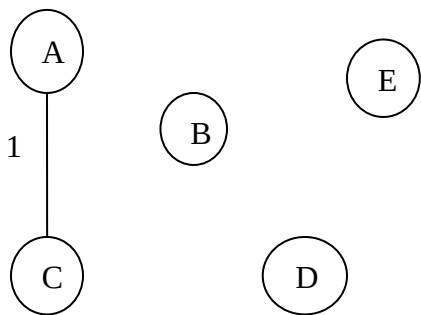
Round Robin Algorithm

This method provides better performance when the number of edges is low. Initially each node is considered to be a partial tree. Each partial tree is maintained in a queue Q. Priority queue is associated with each partial tree, which contains all the arcs ordered by their weights.

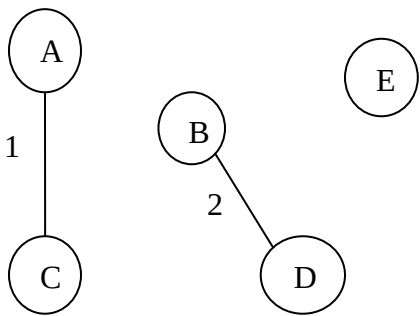
The algorithm proceeds by removing a partial tree, T1, from the front of Q, finding the minimum weight arc a in T1; deleting from Q, the tree T2, at the other end of arc a; combining T1 and T2 into a single new tree T3 and at the same time combining priority queues of T1 and T2 and adding T3 at the rear of priority queue. This continues until Q contains a single tree, the minimum spanning tree.



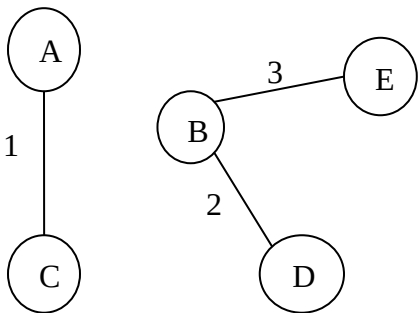
Q	Priority queue
{A}	1, 2
{B}	2, 2, 3, 3
{C}	1, 3, 4
{D}	2, 4, 5
{E}	3, 5



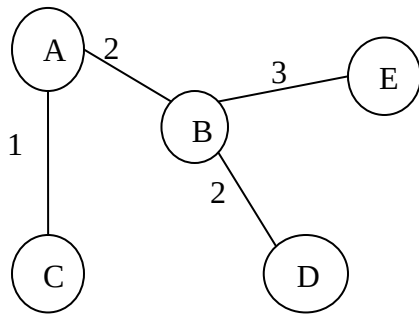
Q	Priority queue
{B}	2, 2, 3, 3
{D}	2, 4, 5
{E}	3, 5
{A, C}	2, 3, 4



Q	Priority queue
{E}	3, 5
{A, C}	2, 3, 4
{B, D}	2, 3, 3, 4, 5



Q	Priority queue
{A, C}	2, 3, 4
{E, B, D}	2, 3, 4, 5, 5



Q	Priority queue
{A, C, E, B, D}	3, 3, 4, 4, 5, 5

Only 1 partial tree is left in the queue, which is the required minimum spanning tree.

Shortest Path Algorithm

There are many problems that can be modeled using graph with weight assigned to their edges. Eg, we may set up the basic graph model by representing cities by vertices and flights by edges. Problems involving distances can be modeled by assigning distances between cities to the edges. Problems involving flight time can be modeled by assigning flight times to edges. Problem involving fares can be modeled by assigning fares to the edges.

Thus, we can model airplane or other mass transit routes by graphs and use shortest path algorithm to compute the best route between two points.

Similarly, if the vertices represent computers; the edges represent a link between computers; and the costs represent communication costs, delay costs, then we can use the shortest-path algorithm to find the cheapest way to send electronic news, data from one computer to a set of other computers.

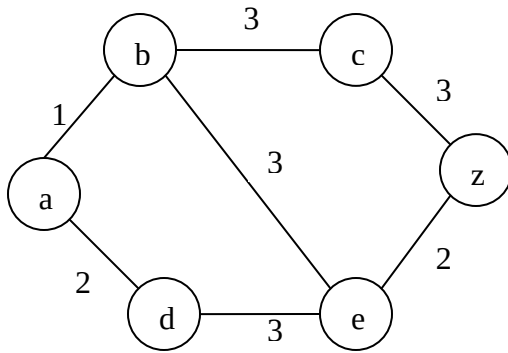
Basically, there are three types of shortest path problems:

Single path: Given two vertices, s and d , find the shortest path from s to d and its length (weights).

Single source: Given a vertex, s find the shortest path to all other vertices.

All pairs: Find the shortest path from all pair of vertices.

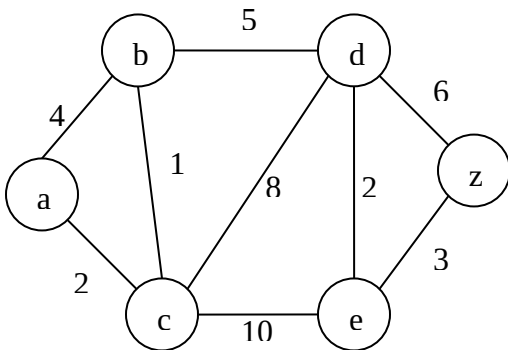
Let source vertex be a. We will find the shortest path to all the other vertices.



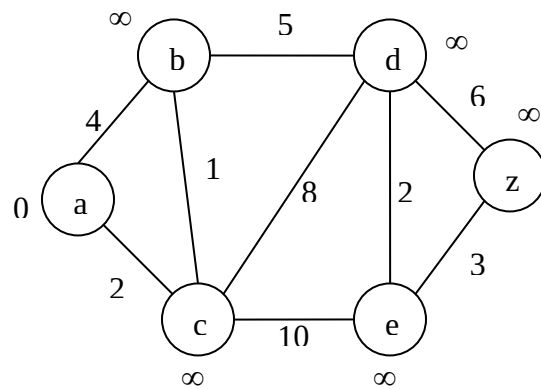
Vertex	Cost	Path
b	1	a, b
c	4	a, b, c
d	2	a, d
e	4	a, b, e
z	6	a, b, e, z

Dijkstra's Algorithm to find the shortest path

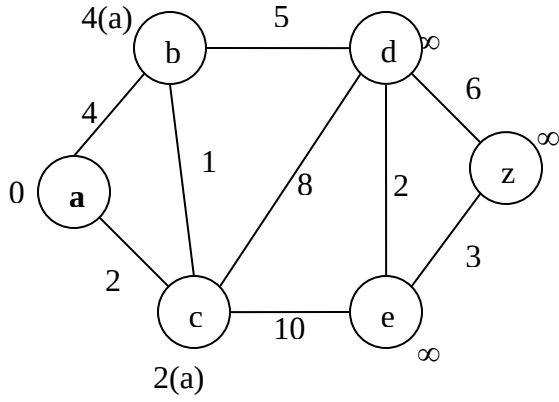
1. Mark all the vertices as unknown
2. for each vertex v keep a distance d_v from source vertex s to v initially set to infinity except for s which is set to $d_s = 0$
3. repeat these steps until all vertices are known
 - i. select a vertex v , which has the smallest d_v among all the unknown vertices
 - ii. mark v as known
 - iii. for each vertex w adjacent to v
 - i. if w is unknown and $d_v + \text{cost}(v, w) < d_w$
update d_w to $d_v + \text{cost}(v, w)$



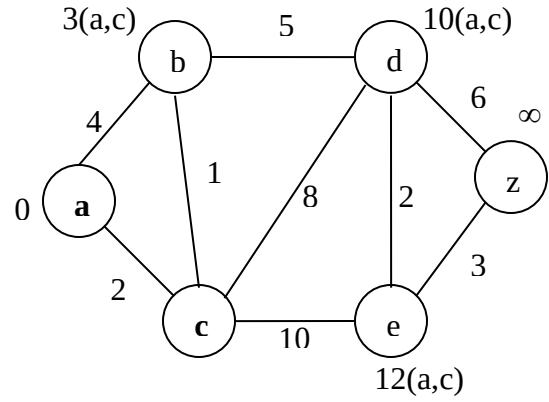
Given graph with weights



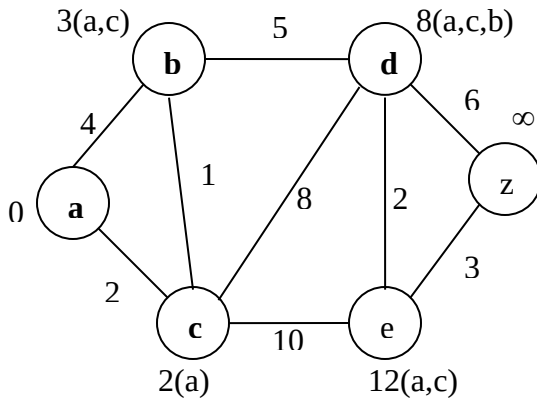
(a)



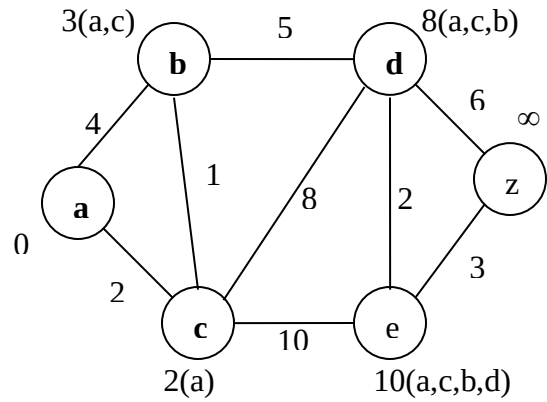
(b)



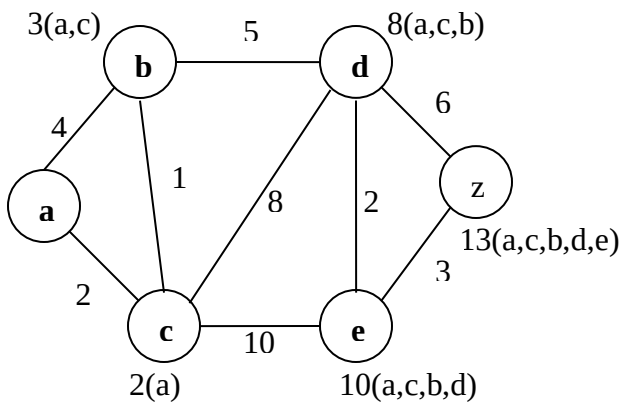
(c)



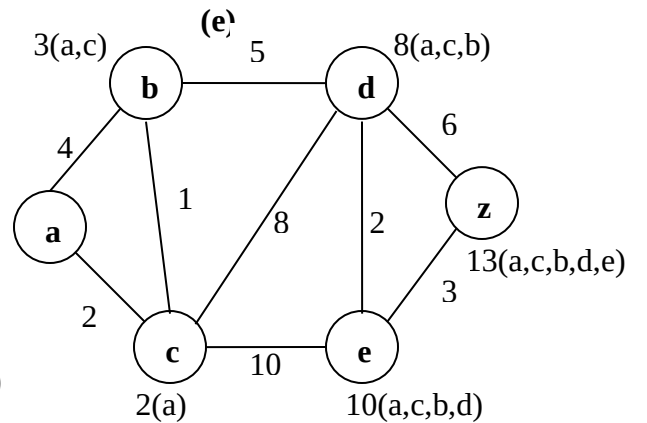
(d)



(e)



(f)



(g)

Fig: Using Dijkstra's Algorithm to find shortest path from a to z